

世界模型部署

LingBot-World

1.3B 图生视频 · Windows 11 原生

单卡 6 GB

Windows 原生

能跑起来吗？

13 帧 · 832×464 · 一步约 12.7 分钟

显存

6 GB

系统内存

64 GB

环境 → 克隆 → 依赖 → 卡点 → 补丁 → 权重 → 13 帧

02 / 实验设计

这次要回答什么

在一台 6GB 显存的 Windows 笔记本上，不装系统级 Python，也不动系统环境，能不能把 LingBot-World 的图生视频跑起来？

官方 README 给的 1.3B 示例是 4 卡多进程，没有单卡 Windows 的安装说明。这次走单进程、单卡的原生 Windows 路径，行不行看这次运行。

项	本次做法
运行环境	Windows 11 原生 PowerShell，不装系统级组件、不改系统环境
Python	由本机已有的 uv 提供 3.12，不装系统级、不改 PATH
文件位置	仓库、依赖、权重、输出都在 F:\AI 一个目录，回滚 = 删目录
模型	只取 1.3B DiT 与共享 T5 / VAE / tokenizer，不下载 14B 本体
本次不测	720p / 60fps、多卡、其他显卡型号、商业用途

这次卡住的地方在安装阶段；下面把报错画面也留着。

这些是本次实验的设计与边界，不代表官方 Windows 支持。

03 / 环境

机器与起点

nvidia-smi: GPU、驱动与驱动自带的 CUDA UMD

on Oct 5 12:10:57 2026

NVIDIA-SMI 610.47										KMD Version: 610.47				CUDA UMD Version: 13.3			
GPU	Name		Driver-Model				Bus-Id	Disp.A	Volatile	Uncorr.	ECC						
Fan	Temp	Perf	Pwr:Usage/Cap				Memory-Usage		GPU-Util	Compute M.	MIG M.						
0	NVIDIA	GeForce RTX 3060	...	WDDM	00000000:01:00.0		Off				N/A						
N/A	53C	P8	10W /	83W	44MiB /		6144MiB	0%		Default	N/A						

系统版本与架构

Caption	Version	OSArchitecture
Microsoft Windows 11 家庭版 中文版	10.0.26200	64 位

机型与系统内存

Manufacturer	Model	TotalRAM_GiB
LENOVO	82RF	63.73

项	本机
GPU / 显存	RTX 3060 Laptop · 6144 MiB
显卡驱动	610.47 (驱动自带 CUDA UMD 13.3)
系统 / 内存 / 目标盘	Windows 11 家庭版 10.0.26200 · 63.73 GiB · F: 第二块 NVMe 剩余 310.3 GiB
已有工具	git 2.55.0 · ffmpeg 9.0.2 (本次不重装)

本机已有的驱动、Git、FFmpeg 只在画面里声明版本；观众要装什么、去哪装，写在手册和简介里。

04 / 环境

代码、解释器与虚拟环境

从零 git clone

```
PS F:\AI> git clone https://github.com/robbyant/lingbot-world-v2.git
Cloning into 'lingbot-world-v2'...
remote: Enumerating objects: 135, done.
remote: Counting objects: 100% (49/49), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 135 (delta 33), reused 13 (delta 13), pack-reused 86 (from 1)
Receiving objects: 100% (135/135), 38.88 MiB | 668.00 KiB/s, done.
Resolving deltas: 100% (52/52), done.
PS F:\AI>
```

01 固定版本

clone 后 HEAD 为 1895d300d8ac936401689b26389f51cbd36530eb, 与预期一致。

02 解释器不装系统级

Python 3.12.13 直接取自本机已有的 uv, 在仓库目录内建 venv; 系统 PATH 不变。

03 虚拟环境

venv 内 pip 26.2.1; 卸载方式就是删掉 F:\AI 这一个目录。

```
Set-Location F:\AI
git clone https://github.com/robbyant/lingbot-world-v2.git
git -C .\lingbot-world-v2 rev-parse HEAD      # 1895d300... (与预期一致)
uv venv .\lingbot-world-v2\.venv --python 3.12
.\lingbot-world-v2\.venv\Scripts\python.exe --version # Python 3.12.13
```

仓库没有 Windows 安装说明, 路径处理和模型加载器都是这次现验的。

命令原文、预期输出与失败处理都写在部署手册里。

05 / 环境

下载：把 2.75 GB 的 torch 从 20 分钟压到 2 分 20 秒

单连接拉 torch 官方 Windows 轮子只有 2.15 MiB/s，
照这个速度要等约 20 分钟。

aria2 多连接下载：CN:16，逐段进度与 ETA

```
aria2 version 1.37.0
PS F:\AI\lingbot-world-v2> aria2c -c -x16 -s16 -k1M --file-allocation=none --
rch.org/whl/cu128/torch-2.11.0%2Bcu128-cp312-cp312-win_amd64.whl'
*** Download Progress Summary as of Mon Oct 05 14:52:59 2026 ***
=====
[#8b0745 398MiB/2.5GiB(15%) CN:16 DL:23MiB ETA:1m34s]
FILE: F:/AI/runtime/wheels/torch-2.11.0+cu128-cp312-cp312-win_amd64.whl
=====

*** Download Progress Summary as of Mon Oct 05 14:53:20 2026 ***
=====
[#8b0745 0.9GiB/2.5GiB(35%) CN:16 DL:24MiB ETA:1m8s]
FILE: F:/AI/runtime/wheels/torch-2.11.0+cu128-cp312-cp312-win_amd64.whl
=====

*** Download Progress Summary as of Mon Oct 05 14:53:41 2026 ***
=====
[#8b0745 1.4GiB/2.5GiB(54%) CN:16 DL:24MiB ETA:48s]
FILE: F:/AI/runtime/wheels/torch-2.11.0+cu128-cp312-cp312-win_amd64.whl
=====
```

curl 单连接

2.15 MiB/s

约 20 分钟

aria2c -c -x16 -s16 -k1M

18 MiB/s

均速；峰值 24 MiB/s，2 分 20 秒完成

做法只有两步：先用 aria2 把轮子抓到本地，再让 pip 从本地文件安装。全程走官方源，不引第三方镜像。

同一条 URL、同一台机器、同一天实测；速度取自下载器自身输出。

06 / 环境

依赖装齐，CUDA 闸门通过

PyTorch CUDA gate: torch / cuda / 版本 / 卡名

```
torch: 2.11.0+cu128
cuda_available: True
torch_cuda: 12.8
gpu: NVIDIA GeForce RTX 3060 Laptop GPU
PS F:\AT\lingbot-world-v2> |
```

包	版本	包	版本
torch	2.11.0+cu128	transformers	4.51.3
torchvision	0.26.0+cu128	diffusers	0.39.0
torchaudio	2.11.0	accelerate	1.15.0
numpy	1.26.4	scipy	1.17.1
opencv-python	4.11.0.86	tokenizers	0.21.4
imageio	2.38.0	einops	0.8.2

A 不能用本机那个 3.14
版本过新，与 torch 的 Windows 轮子对不上；它也只提供 python3 入口。

B 不能用应用商店的别名
python 会落到一个占位程序，建 venv 和装依赖都会失败。

CUDA 闸门的判据是代码里真的返回 True: torch.cuda.is_available() 为 True、torch.version.cuda 为 12.8，卡的名称读得出来。

驱动自带的 CUDA UMD 13.3 与 PyTorch 编译时用的 12.8 不是同一层东西，两者并存不代表冲突。

07 / 卡点

卡点：flash-attn 在 Windows 原生下装不了

官方入口 `pip install flash-attn --no-build-isolation`，在这台机器上退出码 1，import 也失败。

CUDA 版本不匹配，源码编译中止

```
File "C:\AI\lingbot-world-1.3b\src\packages\torch\torch\cpp_extensions.py", line 447, in _check_cuda_version
    raise RuntimeError(CUDA_MISMATCH_MESSAGE, cuda_str_version, torch.version.cuda)
RuntimeError: ('The detected CUDA version (%s) mismatches the version that was used to compile PyTorch (%s). Please make sure t
[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
ERROR: Failed building wheel for flash-attn
Failed to build flash-attn
error: failed-wheel-build-for-install
```

安装退出码 1，接着 import flash_attn 失败

```
PS F:\AI\lingbot-world-v2> $FlashAttnExit = $LASTEXITCODE
PS F:\AI\lingbot-world-v2> Write-Host "flash_attn_install_exit=$FlashAttnExit"
flash_attn_install_exit=1
PS F:\AI\lingbot-world-v2> & $Py -c "import flash_attn; print('flash_attn import ok')"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'flash_attn'
PS F:\AI\lingbot-world-v2> $FlashImportExit = $LASTEXITCODE
PS F:\AI\lingbot-world-v2> Write-Host "flash_attn_import_exit=$FlashImportExit"
flash_attn_import_exit=1
PS F:\AI\lingbot-world-v2>
```

```
Precompiled wheel not found. Building from source...
urllib.error.HTTPError: HTTP Error 404: Not Found
FileNotFoundError: [WinError 2] 系统找不到指定的文件
RuntimeError: The detected CUDA version (13.3) mismatches the version that was used
to compile PyTorch (12.8)
```

三条原因叠在一起：① PyPI 上只有源码包，构建脚本去找预编译轮子得到 HTTP 404；② 本机没有 MSVC，回退源码编译立刻 `FileNotFoundError`；③ 本机检测到 CUDA 13.3，而 PyTorch 是 12.8 编的，版本校验直接抛 `RuntimeError`。

这不是显存问题：失败发生在安装阶段，模型一个字节都还没加载。

解法：给 cross-attention 加一条 PyTorch 等价回退

causal_fast 的 cross-attention 在源码里硬依赖 flash-attn。这里不动依赖，在 attention.py 里补一条等价实现。

```
wan/modules/attention.py (本机实验分支)
① 顶部加 import os
② flash_attention() 开头加分支：没有 flash-attn 时走新实现
③ attention() 改为委托 flash_attention() (原版回退会静默忽略 padding mask)
④ 文件末尾新增 _attention_impl() 与 _sdpa_varlen_attention()
    覆盖 GQA 展开 / 变长 padding mask / causal / 滑动窗口
WAN_ATTENTION_IMPL=auto|flash|sdpa    # 装了 flash-attn 仍优先走 flash-attn
```

验证项	结果
离线数值测试	Ran 17 tests ... OK (17 / 17)
与 float64 手写参考实现比对	逐元素误差 < 2e-5
CUDA 真实形状冒烟	(1, 1560, 12, 128) bfloat16 · 有限值 True · 峰值 23.6 MiB
文件哈希	原版 b6f57f0a... → 补丁版 d0efbca9... (差异 125 行)

这是本机实验改动，不是官方支持的分支；引用它的结论时要说明。

对照测试同时证明：原版官方回退确实会忽略 padding mask，补丁版不会。

09 / 准备

权重：13 项 SHA-256 全部对上

资源	大小	校验
1.3B DiT 六个分片 + index	6.84 GB	7 / 7 匹配
T5 文本编码器 (umt5-xxl bf16)	11.36 GB	匹配
VAE (Wan2.1_VAE.pth)	507.6 MB	匹配
tokenizer 四件套	23.1 MB	4 / 4 匹配
合计	18.73 GB (17.44 GiB)	13 / 13 无缺失、无不匹配

只取这条路径需要的文件，仓库里 74 GB 的 14B 分片一个都没碰。

补装官方依赖清单漏掉的 einops

```
PS F:\AI\lingbot-world-v2> & $Py -m pip install --progress-bar off einops
Requirement already satisfied: einops in .\.venv\Lib\site-packages (0.8.2)
PS F:\AI\lingbot-world-v2> |
```

官方 requirements.txt 里没有 einops，模型代码里却用了 5 处；不补装，generate.py 连 --help 都进不去。这是仓库自己的疏漏，不是环境问题。

大小按字节换算：模型目录 18,729,186,304 字节；13 个文件的 SHA-256 逐个核对。

10 / 运行

13 帧命令与 12.7 分钟

```
python generate.py --task i2v-1.3B --infer_mode causal_fast \  
  --size 480*832 --frame_num 13 --ckpt_dir <DiT> --assets_dir <Assets> \  
  --image examples/03/image.jpg --action_path examples/03 \  
  --prompt "<实测用的英文提示词>" --base_seed 42 \  
  --t5_cpu --convert_model_dtype --offload_model True \  
  --local_attn_size 18 --sink_size 6 --save_file <Out>
```

日志里的生成起点与时间戳



生成开始

16:55:29

写出文件 / 结束

17:08:12 → 17:08:13

从“开始生成”到“写完文件”共约 763 秒，也就是 12 分 43 秒；这段时间里显卡采样利用率 100%。

<p class="note">生成过程不弹预览窗口，脚本只负责把权重载入、逐帧算完、写出文件；屏幕上能看到的只有这个终端。</p>耗时口径 = 模型日志的生成起点到写出文件，含模型加载之后的全过程；不是纯采样，也不是“能实时出片”。

11 / 运行

命令参数逐条

下面是这次实际用的参数，按命令里的顺序。

参数	作用
--task i2v-1.3B	任务与模型规模：图生视频，1.3B
--infer_mode causal_fast	少步蒸馏的因果推理路径
--size 480*832	输入面积与朝向；输出按起始图宽高比改写为 832×464
--frame_num 13	生成帧数；这条路径最短的合法长度是 13 (4n+1)
--ckpt_dir	DiT 权重目录
--assets_dir	T5 / VAE / tokenizer 共享资源目录
--image	起始图（仓库自带 examples/03/image.jpg）
--action_path	相机轨迹动作文件目录；causal_fast 必需
--t5_cpu	T5 文本编码器放在 CPU 上
--convert_model_dtype	加载时把权重转成 bfloat16
--offload_model True	每步前向之后把模型挪回 CPU
--local_attn_size 18	KV cache 的窗口长度
--sink_size 6	KV cache 里固定保留的前缀帧数
--base_seed 42	随机种子
--save_file	输出文件路径

按仓库代码：缺 --action_path 会在采样前退出；--frame_num 小于 13 会把时间维收成 0。

参数名照命令原文写；作用按本次运行日志与仓库代码核对。

12 / 结果

结果：832×464 的 13 帧视频

写出文件与结束：17:08:12 / 17:08:13

```
[2026-10-05 17:08:12,835] INFO: Saving generated video to F:\AI\lingbot-output\lingbot-13f.mp4
[2026-10-05 17:08:13,387] INFO: Finished.
PS F:\AI\lingbot-world-v2> |
```

模型输出视频的第 1、6、13 帧



项	结果
分辨率 / 帧数	832 × 464 · 13 帧
帧率 / 时长	16 fps · 0.8125 秒
文件大小	1,273,344 字节
SHA-256	897c908d04950630a89ac88f95e3831c3327f76 e4be796af0d2b507cf6d66b8f

规格：832×464 / 13 帧 / 16 fps / 0.8125 秒。这条路径用的是本机的 SDPA 回退实现，不是 flash-attn。

成片不到 1 秒；这 13 帧不是实时生成。

13 / 边界

代价与边界

项	本次
attention 实现	本机 SDPA 回退补丁（非官方支持路径）
生成段耗时	约 763 秒（12 分 43 秒）
显存	运行中采样到 4996 MiB / 6144 MiB
输出	832×464 · 13 帧 · 16 fps · 0.8125 秒

显存那格是运行中的一次采样（显卡利用率 100% 时读到 4996 MiB），不是连续峰值，采样间隔也不固定；所以只能写“这次运行中采到 4996 MiB / 6144 MiB，没有 OOM”。

要拿连续峰值得重跑一遍并加监控；本次没有这样做。

这份图文是候审稿：录音稿、配音、字幕、成片与公开发布都还没开始。

官方仓库还有两处会绊人的地方，本次都遇到过：requirements.txt 没写 einops；cross-attention 硬依赖 flash-attn 且没有可用的官方回退。

本次没有验证：720p / 60fps、多卡、其他显卡型号、更大的帧数、真正的实时生成，以及商业用途 —— 模型与代码边界包含 CC BY-NC-SA 4.0。

另外两处缺口：录屏里的 SHA 校验命令写坏了、屏幕上只有报错（13/13 是另行重算的）；模型退出码没能从终端回读。